On the Uncertainty Quantification Ability of Tabular Foundation Models

Tyler R. Johnson, Kian Ben-Jacob, Nima Negarandeh, Oriol Vendrell-Gallart, and Ramin Bostanabad,
University of California, Irvine, CA, 92697, USA

Foundation models (FMs) have achieved substantial success in generalizing across tasks without problem-specific training or fine-tuning. However, many critical applications in mechanics and computational science require not only accurate predictions but also reliable uncertainty quantification (UQ). Herein we investigate the UQ capabilities of tabular FMs in regression tasks through a comprehensive empirical study comparing tabular Prior-Data Fitted Networks (TabPFN) against Gaussian processes (GPs). We systematically evaluate these two methods across a host of regression problems with varying complexity and dataset sizes. We use a default setting to build all of the GPs for a fair comparison against TabPFN v2.5. Our findings highlight an important tradeoff between explicit and learned priors: While TabPFN achieves highly competitive performance for complex, high-dimensional problems with sufficient data, GPs often provide superior predictive accuracy and UQ in data-scarce settings and when the chosen kernel constitutes a good prior Our results can be reproduced from https://github.com/kianswarehouse/GPvsPFN.

Surrogates are increasingly employed to replace or augment expensive simulations. However, selecting the appropriate surrogate type, e.g., a neural network (NN) or a Gaussian process (GP), and training it remains a nontrivial task, particularly when training data are noisy, scarce, or high-dimensional. Consequently, there is growing interest in the engineering and scientific communities in developing probabilistic surrogates, or *emulators*, capable of uncertainty quantification (UQ).

Over the past few decades many emulation techniques have been developed and a nonexhaustive list of the most common ones includes Bayesian NNs,[1] GPs,[2] and ensemble- or dropout- based approaches. These methods provide theoretically strong backbones for UQ and have seen tremendous success in various applications. A feature of all of these "traditional" methods is that they rely on a training or sampling stage. Foundation models (FMs) have emerged as powerful alternatives that aim to dispense with this feature.

FMs are a class of machine learning models that are expected to transfer across tasks with minimal tuning. Beyond providing high accuracy and scalability across different modalities, recent work has shown that transformer-like architectures can approximate Bayesian posterior predictive quantities by learning from samples drawn from a specified prior over datasets. This approximation capability is especially relevant for UQ as it enables probabilistic predictions for new tasks without per task retraining. Building on this idea, tabular FMs, such as tabular Prior-Data Fitted Networks (TabPFN) have emerged as powerful FMs for probabilistically learning from small-to-medium tabular datasets.[3]

These developments motivate a practical and timely study to assess whether FMs, such as TabPFN, deliver uncertainty estimates that are as reliable as those produced by classical methods, such as GPs. Motivated by this need, in this work we empirically evaluate the

1521-9615 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Digital Object Identifier 10.1109/MCSE.2026.3701626
Date of publication 10 June 2026; date of current version 23 September 2026.

Authorized licensed use limited to: Ugandan Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

UQ ability of TabPFN in regression problems by comparing it to GP baselines across different problems, noise levels, dataset sizes, and input dimensionalities (see Figure 1 for some 1-D examples). We focus on reproducible, practitioner-relevant usage by evaluating default implementations throughout, and we measure both predictive accuracy and UQ quality. Our primary contributions are:

- We recommend a principled set of defaults for building GPs to increase their performance across different applications.
- We open source our contributions via GP+[4] and compare its performance to TabPFN v2.5 using three metrics that measure point prediction accuracy (relative root mean squared error, or RRMSE), predictive distribution quality (relative mean negatively oriented interval score, or NIS), and computational costs.
- We discuss limitations of GPs and TabPFN for predictive modeling and UQ, providing a practical guideline for emulator selection.

In the rest of the article we first provide relevant technical background and then elaborate on our comparison methodology. Our findings are summarized in the "Results and Discussion" section, which is followed by the concluding remarks.

TECHNICAL BACKGROUND

TabPFN as a Tabular FM

TabPFN is built on the Prior-Data Fitted Network (PFN) framework, which trains a neural model to approximate the Bayesian posterior predictive distribution induced by a user-defined prior over supervised learning tasks. In this setup, one first specifies a prior over datasets by defining how to sample synthetic tasks. Then, training repeatedly draws datasets from this prior and optimizes the model to predict held-out samples. The PFN training objective is designed such that minimizing it makes the model's predictive distribution close to the true posterior predictive under the specified prior, yielding a Bayesian interpretation: After offline training, the model can output posterior predictive distributions for new datasets through conditioning on the provided context.

Under this view, TabPFN's uncertainty behavior is tightly linked to the designed prior since the model is trained to return predictive distributions consistent with the posterior predictive implied by the synthetic data-generating process. The original TabPFN constructs synthetic tabular tasks from a mixture of generative mechanisms, including causal-style generators and Bayesian NNs, to expose the model to diverse feature–target relationships.

Later versions refine the synthetic-task distribution to better reflect practical tabular issues that affect both accuracy and uncertainty (i.e., incorporating corruptions, such as missing values, uninformative features, and outliers during pretraining).[3] TabPFN v2.0 introduces a randomized feature-token mechanism that handles heterogeneous feature spaces, supporting more reliable transfer of predictions across datasets with differing feature semantics. TabPFN v2.5 further advances the architecture by refining the transformer design and training procedure, resulting in improved predictive accuracy and uncertainty estimates across a broader range of tabular datasets.

Architecturally, TabPFN treats an entire dataset as the input object and uses attention to condition each query prediction on the full labeled context. This dataset-level conditioning means runtime comparisons must distinguish end-to-end inference (conditioning plus prediction) from "traditional" postfit prediction time in conventional fitted models; standard baselines can be faster once fitted, because TabPFN effectively performs dataset conditioning and prediction jointly.[3]

The computational cost of transformer attention scales quadratically in the number of rows processed in a forward pass.[5] For TabPFN-style in-context inference, if N_c is the number of labeled context (training) rows and N_q is the number of query rows, the computational cost scales as

$$\text{Cost} = O\left(N_c\left(N_c + N_q\right)\right) \tag{1}$$

since query rows attend only to training rows and not to each other.[3] Recent versions of TabPFN mitigate this by caching the training-context state, amortizing the $O(N_c^2)$ cost across repeated queries against a fixed training set.

Gaussian Processes

GPs provide a natural Bayesian framework for function approximation by placing a prior on functions and then updating that prior based on data.[2] They are completely characterized by their mean function $m(\boldsymbol{x};\theta)$ and kernel or covariance function $k(\boldsymbol{x},\boldsymbol{x}';\beta)$ where θ and β are hyperparameters that are conventionally estimated based on a training dataset.

The optimal choice of mean and covariance functions depends on the problem but it is common practice to use a zero or constant mean and a stationary

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

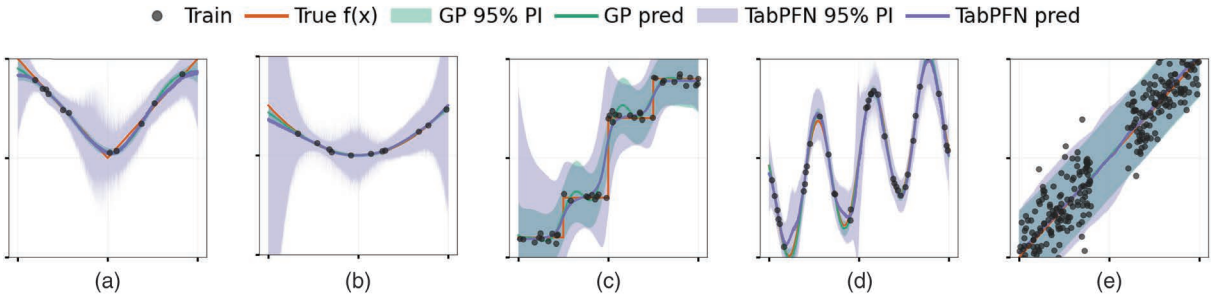


FIGURE 1. One-dimensional examples. The functions are all adopted from Hollman et al.[3] Each plot has the same x and y axes, both spanning [−0.5, 0.5]. All of the GPs are trained on CPU using the defaults of Yousefpour et al.[4] (e.g., constant mean and a Gaussian kernel). TabPFN v2.5 is run on GPU without fine-tuning. (a) $|x|$, (b) x^2, (c) sleep function, (d) $0.3\sin(6.5\pi x) + 0.5x$, and (e) Gappy data.

kernel, such as the Gaussian or Matérn.[6] The power exponential (PE) kernel is another useful stationary kernel, defined as ($\beta = \{s, \omega_i, p\}$ are the hyperparameters)

$$k(\boldsymbol{x}, \boldsymbol{x}'; \beta) = 10^s \exp\left(-\sum_{i=1}^{D_x} 10^{\omega_i} |x_i - x_i'|^p\right) \tag{2}$$

which leverages automatic relevance determination and is equivalent to the Gaussian kernel for $p = 2$.

In practice, regression or interpolation with GPs requires two major steps: 1) selecting the mean and covariance functions, and 2) using the training data for hyperparameter optimization, which is typically done via maximum likelihood estimation. FMs, such as TabPFN, require neither of these steps and hence are very attractive alternatives since the performance of the GP can be sensitive to the selected mean or covariance functions as well as the training process.

GPs have additional limitations, such as scalability, since the cost of repeatedly inverting the covariance matrix during training scales as $O(N^3)$, where N is the number of samples. Their performance also degrades in very high dimensions since selecting an appropriate kernel and effectively estimating its hyperparameters becomes very challenging. Over the past two decades, many effective methods have been developed that address these limitations of GPs (e.g., Wilson et al.[7]) but herein our focus is on "vanilla" GPs that do not leverage any of these advancements.

We trained the GPs via the GP+ package[4] with updated defaults on initialization, formulations based on change of variables, soft constraints, and optimization to produce high-performing emulators. We refer the reader to the GitHub page (https://github.com/kianswarehouse/GPvsPFN) for details and merely provide an example here by noting that the kernel in (2) is commonly formulated as

$$k(\boldsymbol{x}, \boldsymbol{x}'; \beta) = \sigma^2 \exp\left(-\sum_{i=1}^{D_x} \left(\frac{|x_i - x_i'|}{l_i}\right)^p\right) \tag{3}$$

whose hyperparameters are often more challenging to optimize than those in (2).

COMPARISON METHODOLOGY

We designed a set of controlled experiments to compare the emulation power of GPs and TabPFN v2.5 on a host of regression problems. Five of these problems (Wing Weight, Ackley, Dixon-Price, Griewank, and Rosenbrock) are taken from Surjanovic and Bingham,[8] while the Buckling function is adopted from Oune and Bostanabad,[9] to include categorical inputs in our studies. For these six problems, the input dimensionality D_x ranges between 4 and 40 while the output dimensionality is 1 in all problems since TabPFN only accommodates single-response datasets. To include the effect of data size and noise variance on our analyses, for each problem we considered two training dataset sizes, $N = 10D_x$ and $N = 40D_x$, and two noise levels, $\varepsilon \sim \mathcal{N}(0, (0.005 \times c)^2)$ and $\varepsilon \sim \mathcal{N}(0, (0.05 \times c)^2)$, where c is the standard deviation of the target's test data, i.e., c is a problem-dependent constant that adjusts the noise variance based on the problem.

To study the effect of misspecified likelihoods, we also considered non-Gaussian noise in our analytic examples and studied two real-world regression benchmarks, namely, Elevators ($D_x = 18$) and Pumadyn ($D_x = 32$). For evaluation, for each analytic example we generated a single large test dataset of size $N_{\text{test}} = 5000$

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

via a Sobol sequence whose seed number is chosen a priori to ensure TabPFN and GP are compared on the same data. For the two real-world benchmarks, random subsets of the data were used for training and we reserved a single large dataset, $N_{\text{test}} = 3000$, for testing (if the benchmark has insufficient data to train on 20 independent datasets, the subsets were allowed to have some overlap). The median values of our metrics across 20 random sets are reported (see GitHub for statistics).

Let y_i and $\hat{y}_i$ denote the true and predicted values for $i = 1, \ldots, N_{\text{test}}$. RRMSE measures point prediction accuracy and is calculated as

$$\text{RRMSE} = \frac{1}{c}\sqrt{\frac{1}{N_{\text{test}}}\sum_{i=1}^{N_{\text{test}}}(y_i - \hat{y}_i)^2} \tag{4}$$

where c is a problem-dependent scalar (specifically, the response standard deviation in the test data) that ensures RRMSEs are comparable across different problems. To evaluate predictive distribution quality, we used NIS. The negatively oriented interval score for a central $(1-\alpha)$ prediction interval $[L_i, U_i]$ is

$$S_i = \underbrace{(U_i - L_i)}_{a_i} + \underbrace{\frac{(L_i - y_i)\mathbf{1}[y_i < L_i]}{\alpha/2} + \frac{(y_i - U_i)\mathbf{1}[y_i > U_i]}{\alpha/2}}_{b_i}$$

where $\mathbf{1}[\cdot]$ is the indicator function, $\alpha \in (0,1)$ (e.g., $\alpha = 0.05$ for a 95% interval), a_i denotes the interval width, and b_i is the penalty for falling below L_i or above U_i. NIS is then calculated as

$$\text{NIS} = \frac{1}{c}\frac{1}{N_{\text{test}}}\sum_{i=1}^{N_{\text{test}}} a_i + b_i = A + B \tag{5}$$

where c is the same scalar as in (4).

For TabPFN, we construct a central 95% predictive interval by taking the 2.5th and 97.5th percentiles of the predictive distribution at each test input x_i:

$$[L_i, U_i] = [Q_{0.025}(y|x_i), Q_{0.975}(y|x_i)].$$

Because this interval is quantile-based, it need not be symmetric about the predictive mean. For GPs, we report a Gaussian 95% interval using the predictive mean μ_i and standard deviation σ_i.

We emphasize that, unlike TabPFN, GPs require training, which typically requires some fine-tuning on the selection of the kernel, hyperparameter priors, and optimization settings. However, we refrain from this fine-tuning and train the GPs using the *defaults* of our GP+ package.[4] Obviously, this choice indicates that our GPs will *not* be the best (or maybe even close to the best) GPs one can build, but we have made this choice since the competing method (i.e., TabPFN) does not require the user to spend any time on training. Additionally, TabPFN does not require any preprocessing. So, we primarily rely on the default preprocessing of GP+, which scales each input to [0, 1] and standardizes the output vector based on its mean and variance.

Training and prediction were done on CPU for GPs, but since TabPFN is a transformer-based model, we use it on GPU (TabPFN provides almost identical performance metrics on CPU but at a significantly higher wall clock time) (see Table 1). The hardware used in these experiments was the 11th Gen Intel Cor i7-11700K @ 3.6 GHz and the NVIDIA GeForce RTX 3060 (12GB) for CPU and GPU, respectively.

RESULTS AND DISCUSSIONS

The main results of our studies are summarized in Table 1 and Figure 2, where the latter only visualizes a subset of Table 1 due to space constraints. Note that while the medians across 20 independent runs are representative; individual runs may show different relative rankings between the two models on either RRMSE or NIS (see GitHub for details). We analyze these results below in terms of accuracy, UQ, cost, and ease-of-use of the models. We note that we also experimented with adjusting multiple TabPFN configuration settings, but observed no meaningful improvement over the defaults, suggesting that out-of-the-box TabPFN already performs near its ceiling on these benchmarks.

Accuracy and UQ

Our findings suggest that neither of the methods outperforms the alternative consistently. The overall trends are also consistent in that as dataset size is increased or noise variance is reduced their performance improves and the relative rankings are mostly unchanged.

This outcome seems counterintuitive at first as one may expect a "trained" model to outperform, but as explained below it is primarily due to the fact that the GPs in Table 1 are trained via default settings of GP+ (e.g., Gaussian kernel). For example, in the Wing Weight benchmark in Figure 2(a), GP+ achieves very high emulation accuracy where the test RRMSE approaches the injected noise levels (e.g., 0.005 and 0.05). GP+ achieves lower RRMSE on Wing Weight across all reported settings in Table 1, and generally lower NIS, with TabPFN marginally outperforming GP+ on NIS only in the higher-noise, small-data setting. In the lower-noise setting, TabPFN's RRMSE and NIS distributions remain

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. Summary of results.

				GP+			TabPFN v2.5		
Problem	**N**	**D_x**	**Noise**	**RRMSE**	**NIS (A + B)**	**CPU (s)**	**RRMSE**	**NIS (A + B)**	**GPU (CPU) (s)**
Wing Weight	10 D_x	10	0.005	**0.016**	**0.073** (0.05 + 0.023)	**1.07**	0.033	0.167 (0.164 + 0.004)	1.36 (21.98)
	10 D_x	10	0.05	**0.068**	0.331 (0.252 + 0.079)	**1.11**	0.07	**0.327** (0.294 + 0.034)	1.38 (21)
	40 D_x	10	0.005	**0.007**	**0.033** (0.026 + 0.007)	7.44	0.013	0.082 (0.082 + 0.0)	**1.5** (24.55)
	40 D_x	10	0.05	**0.055**	**0.257** (0.212 + 0.045)	7.90	0.057	0.269 (0.224 + 0.045)	**1.45** (24.54)
Buckling	10 D_x	4	0.005	**0.174**	1.567 (0.044 + 1.523)	9.64	0.419	**0.656** (0.655 + 0.001)	**1.1** (13.37)
	10 D_x	4	0.05	**0.304**	3.384 (0.203 + 3.181)	4.97	0.323	**1.035** (0.997 + 0.037)	**1.09** (13.24)
	40 D_x	4	0.005	**0.016**	**0.056** (0.025 + 0.031)	11.88	0.097	0.155 (0.153 + 0.002)	**1.06** (14.18)
	40 D_x	4	0.05	**0.078**	**0.36** (0.215 + 0.145)	6.42	0.231	0.469 (0.430 + 0.039)	**1.07** (13.93)
Ackley	10 D_x	10	0.005	**0.301**	**1.569** (1.297 + 0.271)	**1.16**	0.378	1.887 (1.787 + 0.1)	1.38 (21.11)
	10 D_x	10	0.05	**0.31**	**1.616** (1.296 + 0.321)	**1.13**	0.383	1.938 (1.82 + 0.117)	1.39 (21.49)
	40 D_x	10	0.005	0.213	1.073 (0.881 + 0.192)	7.61	**0.21**	**0.993** (0.791 + 0.202)	**1.48** (25.32)
	40 D_x	10	0.05	0.22	1.112 (0.92 + 0.192)	7.88	**0.217**	**1.036** (0.818 + 0.218)	**1.48** (23.62)
	10 D_x	20	0.005	**0.316**	**1.614** (1.231 + 0.383)	3.37	0.385	1.830 (1.709 + 0.121)	**1.96** (32.07)
	10 D_x	20	0.05	**0.321**	**1.621** (1.234 + 0.387)	3.42	0.39	1.866 (1.745 + 0.121)	**1.97** (31.44)
	40 D_x	20	0.005	0.217	1.069 (0.901 + 0.168)	59.54	**0.214**	**0.995** (0.844 + 0.151)	**2.35** (45.3)
	40 D_x	20	0.05	0.225	1.096 (0.921 + 0.175)	60.42	**0.222**	**1.037** (0.877 + 0.16)	**2.36** (45.18)
	10 D_x	40	0.005	**0.298**	**1.495** (1.265 + 0.230)	18.00	0.416	1.962 (1.794 + 0.168)	**3.2** (62.85)
	10 D_x	40	0.05	**0.307**	**1.531** (1.281 + 0.25)	17.91	0.421	1.987 (1.81 + 0.177)	**3.2** (62.99)
	40 D_x	40	0.005	0.243	1.182 (0.922 + 0.26)	467.44	**0.23**	**1.114** (1.003 + 0.111)	**4.83** (119.17)
	40 D_x	40	0.05	0.251	1.202 (0.944 + 0.259)	479.8	**0.237**	**1.138** (1.026 + 0.111)	**4.85** (118.15)
Dixon-Price	10 D_x	10	0.005	0.442	2.037 (1.539 + 0.498)	**1.24**	**0.427**	**2.010** (1.854 + 0.157)	1.42 (18.84)
	10 D_x	10	0.05	0.453	2.085 (1.521 + 0.564)	**1.25**	**0.431**	**2.057** (1.884 + 0.173)	1.43 (19.04)
	40 D_x	10	0.005	0.279	1.287 (1.049 + 0.237)	11.46	**0.162**	**0.840** (0.79 + 0.05)	**1.53** (21.37)
	40 D_x	10	0.05	0.288	1.325 (1.08 + 0.245)	11.4	**0.177**	**0.873** (0.815 + 0.059)	**1.51** (21.79)

(*Continued*)

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. (*Continued.*)

				GP+			TabPFN v2.5		
Problem	**N**	D_x	**Noise**	**RRMSE**	**NIS (*A* + *B*)**	**CPU (s)**	**RRMSE**	**NIS (*A* + *B*)**	**GPU (CPU) (s)**
	10 D_x	20	0.005	0.474	2.286 (1.553 + 0.733)	4.78	**0.469**	**2.159** (1.911 + 0.247)	**1.96** (32.29)
	10 D_x	20	0.05	**0.466**	2.227 (1.569 + 0.658)	4.45	0.471	**2.172** (1.915 + 0.257)	**1.97** (32.73)
	40 D_x	20	0.005	0.334	1.558 (1.3 + 0.258)	99.93	**0.195**	**0.977** (0.91 + 0.066)	**2.37** (44.96)
	40 D_x	20	0.05	0.339	1.561 (1.329 + 0.233)	99.77	**0.203**	**1.006** (0.93 + 0.076)	**2.36** (46.18)
	10 D_x	40	0.005	**0.455**	**2.183** (1.618 + 0.565)	31.05	0.48	2.327 (2.009 + 0.318)	**3.2** (62.67)
	10 D_x	40	0.05	**0.47**	**2.24** (1.635 + 0.605)	32.35	0.482	2.33 (2.015 + 0.315)	**3.31** (62.69)
	40 D_x	40	0.005	0.431	2.086 (1.481 + 0.605)	867.45	**0.231**	**1.141** (1.066 + 0.075)	**4.80** (117.45)
	40 D_x	40	0.05	0.433	2.040 (1.491 + 0.549)	874.93	**0.237**	**1.163** (1.086 + 0.078)	**4.85** (119.56)
Griewank	10 D_x	10	0.005	**0.011**	**0.111** (0.111 + 0.0)	**1.22**	0.512	2.460 (2.27 + 0.19)	1.37 (18.52)
	10 D_x	10	0.05	**0.109**	**0.529** (0.482 + 0.047)	**1.13**	0.524	2.49 (2.281 + 0.209)	1.37 (19.48)
	40 D_x	10	0.005	**0.007**	**0.033** (0.028 + 0.005)	12.01	0.1	0.469 (0.444 + 0.024)	**1.45** (21.11)
	40 D_x	10	0.05	**0.056**	**0.262** (0.219 + 0.042)	11.68	0.115	0.535 (0.479 + 0.055)	**1.44** (21.67)
	10 D_x	20	0.005	**0.139**	**1.009** (1.009 + 0.0)	3.77	0.515	2.426 (2.175 + 0.251)	**1.96** (32.05)
	10 D_x	20	0.05	**0.174**	**1.065** (1.06 + 0.005)	3.80	0.516	2.436 (2.176 + 0.261)	**1.95** (32.14)
	40 D_x	20	0.005	**0.008**	**0.04** (0.035 + 0.005)	88.19	0.14	0.644 (0.6 + 0.044)	**2.37** (44.84)
	40 D_x	20	0.05	**0.061**	**0.282** (0.242 + 0.04)	91.18	0.149	0.691 (0.629 + 0.062)	**2.37** (45.61)
	10 D_x	40	0.005	**0.255**	**1.552** (1.543 + 0.009)	23.33	0.548	2.549 (2.206 + 0.343)	**3.19** (62.62)
	10 D_x	40	0.05	**0.257**	**1.562** (1.553 + 0.009)	23.07	0.55	2.585 (2.224 + 0.361)	**3.19** (62.97)
	40 D_x	40	0.005	**0.008**	**0.047** (0.047 + 0.0)	824.65	0.173	0.970 (0.901 + 0.068)	**4.82** (118.7)
	40 D_x	40	0.05	**0.074**	**0.347** (0.299 + 0.048)	837.12	0.181	1.007 (0.929 + 0.077)	**4.83** (118.21)
Rosenbrock	10 D_x	10	0.005	0.459	2.206 (1.622 + 0.584)	**1.11**	**0.379**	**2.084** (2.015 + 0.069)	1.39 (19.12)
	10 D_x	10	0.05	0.462	2.285 (1.655 + 0.63)	**1.17**	**0.386**	**2.079** (2.01 + 0.069)	1.39 (18.61)
	40 D_x	10	0.005	0.337	1.588 (1.194 + 0.394)	8.7	**0.105**	**0.5** (0.467 + 0.033)	**1.49** (21.92)
	40 D_x	10	0.05	0.34	1.617 (1.226 + 0.391)	8.74	**0.122**	**0.567** (0.517 + 0.051)	**1.5** (22.94)
	10 D_x	20	0.005	0.49	2.435 (1.643 + 0.792)	3.99	**0.468**	**2.289** (2.157 + 0.132)	**2.03** (31.58)

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Only. Restrictions apply. 2026

TABLE 1. (*Continued.*)

				GP+			TabPFN v2.5		
Problem	**N**	D_x	**Noise**	**RRMSE**	**NIS (A + B)**	**CPU (s)**	**RRMSE**	**NIS (A + B)**	**GPU (CPU) (s)**
	10 D_x	20	0.05	0.492	2.395 (1.668 + 0.727)	3.93	**0.475**	**2.302** (2.163 + 0.139)	**1.92** (32.09)
	40 D_x	20	0.005	0.373	1.752 (1.428 + 0.324)	70.29	**0.157**	**0.764** (0.723 + 0.041)	**2.34** (54)
	40 D_x	20	0.05	0.377	1.774 (1.443 + 0.331)	66.62	**0.165**	**0.803** (0.749 + 0.054)	**2.35** (48.69)
	10 D_x	40	0.005	**0.481**	**2.281** (1.683 + 0.598)	21.93	0.501	2.371 (2.123 + 0.248)	**3.19** (62.61)
	10 D_x	40	0.05	**0.484**	**2.297** (1.685 + 0.612)	21.65	0.503	2.406 (2.152 + 0.255)	**3.19** (62.87)
	40 D_x	40	0.005	0.422	1.997 (1.492 + 0.505)	592.45	**0.215**	**1.071** (0.991 + 0.081)	**4.83** (118.52)
	40 D_x	40	0.05	0.427	2.028 (1.502 + 0.525)	578.19	**0.222**	**1.099** (1.012 + 0.086)	**4.82** (118.95)

RRMSE, NIS, and wall-clock time are reported (medians across 20 runs, see GitHub for statistics). Best numbers are in bold font and noise refers to either $\varepsilon \sim \mathcal{N}(0, (0.005 \times c)^2)$ or $\varepsilon \sim \mathcal{N}(0, (0.05 \times c)^2)$, where c is a problem-dependent constant that adjusts the noise variance based on the problem (see GitHub for non-Gaussian noise). All GPs use default GP+ settings. TabPFN runs on GPU (CPU time in parentheses) and the compute time is almost entirely for inference. GP runs on CPU with the majority of its compute time spent during training.

shifted upward relative to GP+, suggesting it does not fully exploit the near-deterministic regime as effectively as the GP baseline. Another notable behavior in this regime is that TabPFN's NIS distribution remains relatively tight across runs even when its RRMSE varies substantially. This connotes that TabPFN's predictive intervals remain fairly well-behaved across splits, but its predictive mean can be less accurate than the uncertainty score would imply.

In the Buckling example, TabPFN v2.5 exhibits considerably higher variability in RRMSE than GP+ across all reported settings, reflecting sensitivity of its mean predictions to the specific training split when context is very small. Despite unreliable mean predictions, TabPFN produces substantially better-calibrated intervals than GP+ at $N = 10D_x$. As N increases to $40D_x$, GP+ improves dramatically in both RRMSE and NIS, while TabPFN's gains are comparatively modest. In Figure 2(b), TabPFN's RRMSE remains highly variable across runs while its NIS stays comparatively tight and well-behaved, suggesting TabPFN maintains appropriate predictive uncertainty even in runs where its median predictions are poor. Testing at higher data volumes (results not shown) confirmed that GP+ improves more efficiently with additional data than TabPFN, whose accuracy stagnates in this case.

Ackley is a highly multimodal benchmark for which both methods achieve relatively high RRMSEs and NISs at $N = 10D_x$, with TabPFN typically exhibiting greater run-to-run variability and higher medians than GP+. When the training set increases to $N = 40D_x$, both models improve and the two methods become comparable: TabPFN often achieves slightly lower median RRMSE and NIS, while GP+ tends to retain tighter distributions across runs; see Figure 2(c) for a subset of results. We note that our studies are limited to $N \leq 40D_x$ for high-dimensional problems as we refrain from using scalable GP methods. Comparison of TabPFN to scalable GPs is left for future work.

The Griewank benchmark highlights a clear advantage for GP+. In the low-data regime ($N = 10D_x$), TabPFN v2.5 exhibits substantially higher RRMSE and NIS, indicating difficulty learning the underlying structure of the objective from limited context. As the dataset size increases to $N = 40D_x$, GP+ improves sharply and reaches very low RRMSE and NIS, while TabPFN improves only modestly and remains shifted upward in both metrics. Griewank illustrates a regime where the GP prior and default training procedure yield strong sample efficiency and stable uncertainty estimates, whereas TabPFN requires substantially more data to become competitive (see Table 1).

We observe interesting trends in Dixon-Price and Rosenbrock benchmarks, where GP+ outperforms or is comparable to TabPFN in low-data high-dimensional regimes (e.g., $N = 10D_x$ and $D_x = 40$) but loses advantage as N increases to the extent that, unlike TabPFN, little gains are obtained in both RRMSE and NIS. As discussed in the following, these counterintuitive trends are due to the fixed GP+ defaults used across all problems.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

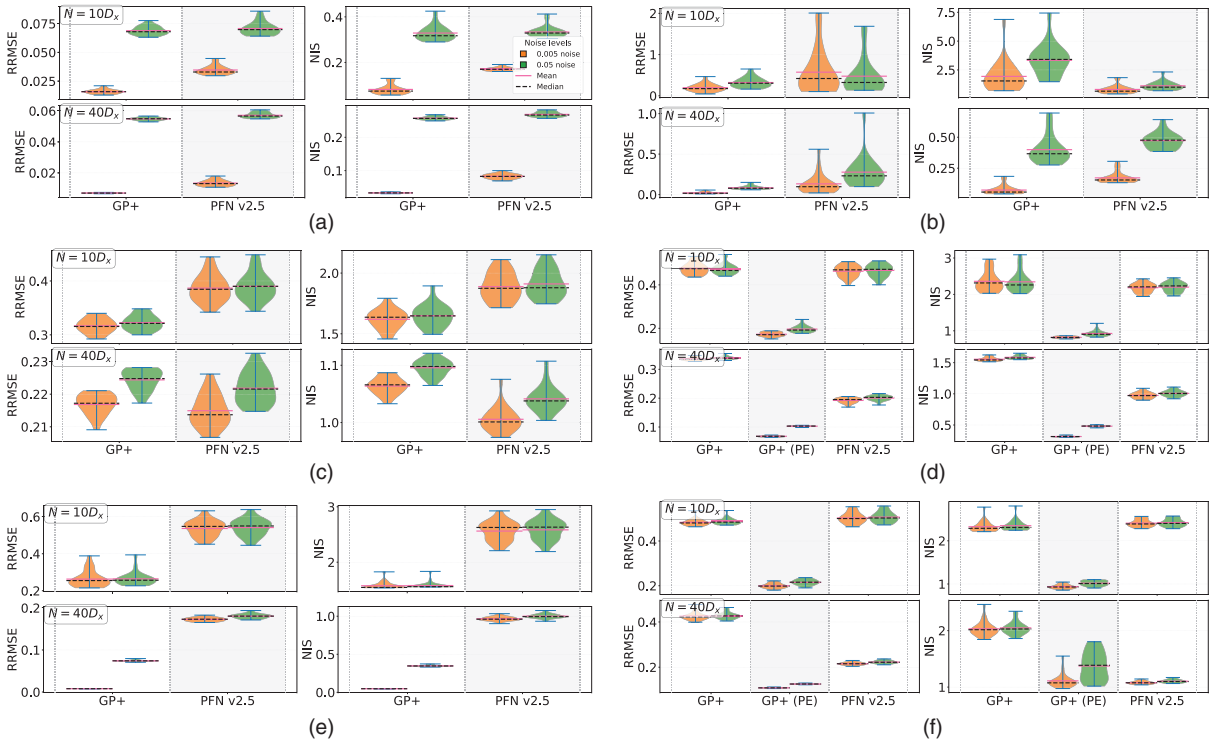


FIGURE 2. Visualization of a subset of simulations listed in Table 1. RRMSE and NIS are used to compare TabPFN and GP across six benchmarks whose input dimensionality, D_x, ranges from 4 to 40 (only Buckling includes categorical inputs). Size of the training data is either $10D_x$ or $40D_x$ and noise is either $\varepsilon \sim \mathcal{N}(0, (0.005 \times c)^2)$ or $\varepsilon \sim \mathcal{N}(0, (0.05 \times c)^2)$. For Dixon-Price and Rosenbrock, we also consider the PE kernel to show the effect of kernel type on GPs. All of the GPs are trained with the *default* settings of GP+ package.[4] GPs are used on CPU while TabPFN is used on GPU. (a) Wing Weight: $D_x = 10$, (b) Buckling: $D_x = 4$, (c) Ackley: $D_x = 20$, (d) Dixon-Price: $D_x = 20$, (e) Griewank: $D_x = 40$, and (f) Rosebrock: $D_x = 40$.

Effect of Kernel and Mean Choice on GP

There are two cases shown in Figure 2(d) and (f) where, unlike TabPFN, the performance of GP + does not improve noticeably as more samples are used in training. Normally, one leverages the information gained from the GP's interpretable hyperparameters or loss function value (i.e., log marginal likelihood) to implement different strategies that can enhance the models. These techniques increase the cost and complexity of training GPs, which TabPFN users do not typically have to use.

We show in Figure 2(d) and (f) that simply changing the kernel from Gaussian to PE significantly improves the performance of the GP. It is important to note that this adds another hyperparameter and as a result increases the training cost. Additionally, when training GPs on high-dimensional (e.g., $D_x = 40$) versions of the Rosenbrock function, we observed that the estimate of the constant mean depended substantially on the optimization initialization. Based on this observation, we experimented with a zero-mean GP that performed much better. Since we do not use scalable GP approaches, we reduced the number of initializations during hyperparameter optimization from the default 16 to 8, which reduced training costs but may have contributed to a higher overall NIS. This approach indicates that by paying the price of fine-tuning the model setup and training for a particular application, GPs can provide highly accurate results.

Computational Cost

Table 1 reports the combined training and prediction time for GP+ and end-to-end inference time (conditioning plus prediction) for TabPFN v2.5, where for GP+ the majority of the cost is spent during training, while for TabPFN it is spent during prediction. These timings are hardware-dependent because we run the GPs on CPU and TabPFN on GPU, so they should be

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Only. Restrictions apply.

TABLE 2. Summary of results for real-world datasets. RRMSE, NIS, and wall-clock time (medians across 20 runs, see GitHub for statistics). TabPFN reports GPU time with CPU time in parentheses. GP runs on CPU; TabPFN on GPU. All GPs use default GP+ settings.

			GP+			TabPFN v2.5		
Problem	**N**	D_x	**RRMSE**	**NIS ($A+B$)**	**CPU Time (s)**	**RRMSE**	**NIS ($A+B$)**	**GPU Time (s)**
Elevators	10 D_x	18	0.381	2.060 (1.292 + 0.768)	2.4	**0.336**	**1.591** (1.274 + 0.317)	**0.97**
	20 D_x	18	0.347	1.827 (1.275 + 0.552)	9.43	**0.302**	**1.413** (1.107 + 0.306)	**1.06**
	40 D_x	18	0.324	1.685 (1.245 + 0.413)	43.62	**0.288**	**1.327** (1.029 + 0.297)	**1.23**
Pumadyn	10 D_x	32	**0.206**	**0.971** (0.762 + 0.209)	13.69	0.257	1.202 (0.952 + 0.25)	**1.6**
	20 D_x	32	**0.194**	**0.9** (0.736 + 0.164)	71.44	0.252	1.162 (0.919 + 0.243)	**1.89**
	40 D_x	32	**0.186**	**0.95** (0.865 + 0.085)	423.25	0.25	1.137 (0.907 + 0.231)	**2.47**

Best numbers are in bold.

interpreted as representative wall-clock behavior rather than a device-normalized comparison. For reference, Table 1 also reports TabPFN's wall-clock time on CPU (in parentheses), which is substantially higher than its GPU time and highlights the importance of dedicated hardware for transformer-based inference.

GP+ training time is not solely a function of N and D_x; it also depends on how easily the log marginal-likelihood landscape can be optimized for a given problem. For example, on Wing Weight with $D_x = 10$ and $N = 10D_x$, GP+ completes training and prediction in roughly 1.1 s, whereas Buckling at the same training-size scaling takes 5–10 s depending on the noise level, despite having fewer total training samples ($N = 40$ versus $N = 100$), with the low-noise setting being slower due to a flatter likelihood surface that is harder to optimize. Despite this, TabPFN's end-to-end inference cost is roughly constant at ≈1.4 s on GPU across these cases. As N and D_x grow, exact GP training quickly dominates: on Griewank with $D_x = 40$ and $N = 40D_x$, GP+ takes $\approx 8.3 \times 10^2$ s, whereas TabPFN finishes in ≈4.8 s on GPU. This pattern is consistent across the high-dimensional $N = 40D_x$ cases.

A key detail is that our reported prediction times are measured on a large test set ($N_{\text{test}} = 5000$). If only a small number of queries were needed per fit (e.g., under 100), TabPFN's end-to-end inference time would drop noticeably since the test set substantially outsizes the training context ($N_c \leq 10D_x$) in our setup.

These tradeoffs matter most in downstream use. In Bayesian optimization, we often fit once and score many candidate points, so fast postfit GP prediction is an advantage when training remains affordable. Conversely, when the dataset is updated frequently and only a small number of predictions are needed per update, TabPFN can be attractive because it avoids per-dataset hyperparameter optimization.

Additional Studies and Practical Implications

To assess robustness beyond the synthetic Gaussian-noise setting, we additionally evaluated both methods under Student's t-distributed noise ($\nu = 4$) and on two real-world tabular regression datasets (Elevators and Pumadyn). The aforementioned qualitative trends were preserved in both cases, suggesting our conclusions are not an artifact of the Gaussian observation model that favors GPs. Full tables and figures are provided in our GitHub repository.

In Table 2, we see that, with respect to their median performance, TabPFN outperforms GP+ in the Elevators problem and the opposite is observed in the Pumadyn problem. It is important to note that the performance of both models does not improve much as the training dataset increases in size, and the gap between the median performance remains consistent as more data are added to the training set. Similar to Table 1, we observe that the method with the best RRMSE also provides the best NIS, indicating that both methods provide decent UQ.

Across the benchmarks, GP+ with a single default configuration is often competitive with TabPFN v2.5, but the performance depends on how well the GP prior matches the function class and on problem difficulty (e.g., high dimensionality and strong multimodality). In very low-noise settings, GP+ more reliably tracks the noise floor, while TabPFN tends to retain a residual error even as more data are provided. TabPFN seems less suitable for high-precision applications that

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.

require errors near the simulator noise floor. Conversely, when the default GP kernel/mean is misspecified, TabPFN can narrow the gap or surpass GP+ as context grows. Practically, this suggests using GP+ defaults as a strong baseline and applying minor adjustments to the kernel/mean when performance saturates, before resorting to heavier tuning.

CONCLUSION AND FUTURE DIRECTIONS

We compared the performance of tabular PFNs and GPs on a set of benchmarks. TabPFN offers a computationally cheap alternative to traditional emulators that is able to generalize to many datasets in high-dimensional, large-data, and mixed-input regimes. It is becoming an increasingly popular method due to its ability to reduce the computational overhead required from fitting a good emulator. This FM can remove the need to manually preprocess a dataset or train and tune a model. Our evaluation of GPs and TabPFN showed that TabPFN is a cost-effective method to obtain accurate point estimates, but its ability does not tend to rival GPs if they are equipped with a good kernel.

We observed that increasing the dataset size in some of the benchmarks (i.e., Dixon-Price and Rosenbrock) negligibly improved performance while dramatically increasing the training costs of GPs. Meanwhile, TabPFN was able to leverage more training samples to approximate the Bayesian posterior predictive distribution better and this shows its ability to scale well with larger datasets. This trend is due to the fact that all of our GPs were trained with default settings of GP+, while it is common practice to fine-tune the mean function or kernel of the GP slightly based on the application. As shown in our results, fine-tuning can substantially improve the performance of GPs but does require inspecting the hyperparameters and the loss function. In contrast, our attempts to tune TabPFN beyond its defaults did not yield meaningful improvements, indicating that the default configuration is already a strong operating point and that the user effort saved on the TabPFN side is unlikely to be recovered through tuning.

From an uncertainty perspective, both methods can produce reasonable intervals, but fail differently and offer different levels of interpretability. GP behavior can often be diagnosed through learned hyperparameters (e.g., noise and lengthscales), and GP+ exposes a direct decomposition of uncertainty: The learned likelihood noise (nugget) corresponds to aleatoric uncertainty, while the posterior variance of the latent function reflects epistemic uncertainty. TabPFN also outputs a flexible predictive distribution, but its uncertainty is produced by a NN and is not explicitly parameterized into observation-noise and latent-function components.

Our studies excluded recent advances on TabPFN and on GPs, including kernel construction and log marginal-likelihood optimization in high-dimensional, complex, or large-data settings.[10,11,12] Also, the complexity of our benchmarks was limited to two real-world datasets and six analytic problems with Gaussian or Students' *t*-distribution noise. Additionally, while RRMSE and NIS are useful metrics for assessing emulation performance, we believe more insights can be obtained via application-oriented comparisons, such as Bayesian optimization, multifidelity modeling, or multitask emulation. These directions will be pursued in our future research.

ACKNOWLEDGMENTS

We appreciate the support from the Office of the Naval Research (Award N000142312485) and National Science Foundation (Award 2238038).

REFERENCES

1. C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural network," in *Proc. 32nd Int. Conf. Mach. Learn., (PMLR)*, 2015, pp. 1613–1622.
2. C. Rasmussen and C. Williams, *Gaussian Processes for Machine Learning*, Cambridge, MA, USA: MIT Press, 2006.
3. N. Hollmann et al., "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, no. 8045, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6.
4. A. Yousefpour, Z. Z. Foumani, M. Shishehbor, C. Mora, and R. Bostanabad, "Gp+: A Python library for kernel-based learning via Gaussian processes," *Adv. Eng. Softw.*, vol. 195, Sep. 2024, Art. no. 103686, doi: 10.1016/j.advengsoft.2024.103686.
5. T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," 2021, *arXiv:2106.04554*.
6. M. G. Genton, "Classes of kernels for machine learning: A statistics perspective," *J. Mach. Learn. Res.*, vol. 2, pp. 299–312, Dec. 2001.
7. A. G. Wilson, Z. Hu, R. Salakhutdinov, and E. P. Xing, "Deep kernel learning," in *Proc. 19th Int. Conf. Artif. Intell. Statist., (PMLR)*, 2016, pp. 370–378.
8. S. Surjanovic and D. Bingham, "Virtual Library of Simulation Experiments: Test Functions and Datasets," Accessed: Jan. 23, 2026. [Online]. Available: http://www.sfu.ca/~ssurjano

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Only. Restrictions apply.

9. N. Oune and R. Bostanabad, "Latent map Gaussian processes for mixed variable metamodeling," *Comput. Methods Appl. Mech. Eng.*, vol. 387, Dec. 2021, Art. no. 114128, doi: 10.1016/j.cma.2021.114128. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S004578252100459X
10. N. Negarandeh, C. Mora, and R. Bostanabad, "Non-stationary kernel learning in Gaussian processes," *J. Mech. Des.*, vol. 148, no. 2, pp. 1–11, 2026, doi: 10.1115/1.4070614.
11. A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Proc. 21th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2007, pp. 1177–1184.
12. M. Binois and N. Wycoff, "A survey on high-dimensional Gaussian process modeling with application to Bayesian optimization," *ACM Trans. Evol. Learn. Optim.*, vol. 2, no. 2, pp. 1–26, 2022, doi: 10.1145/3545611.

TYLER R. JOHNSON is a Ph.D. student at the University of California, Irvine, Irvine, CA, 92697, USA. His research interests include uncertainty quantification, multifidelity emulation, and calibration. Johnson received his B.S. degree in aerospace engineering from the University of California, Irvine. He is a member of the National Society of Black Engineers. Contact him at trjohns1@uci.edu.

KIAN BEN-JACOB is a Ph.D. student at the University of California, Irvine, Irvine, CA, 92697, USA. His research interests include multifidelity emulation and Bayesian optimization. Ben-Jacob received his B.S. degree in mechanical engineering from the University of Utah. He is a member of the American Institute of Aeronautics and Astronautics. Contact him at kbenjaco@uci.edu.

NIMA NEGARANDEH is a Ph.D. student at the University of California, Irvine, Irvine, CA, 92697, USA. His research interests include scientific machine learning, design under uncertainty, and probabilistic modeling. Negarandeh received his M.S. degree in mechanical engineering from the University of Tehran. Contact him at nnegaran@uci.edu.

ORIOL VENDRELL-GALLART is a Ph.D. student at the University of California, Irvine, Irvine, CA, 92697, USA. His research interests include operator learning, foundation models and scientific machine learning. Vendrell-Gallart received his M.S. degree in automatic control from the Polytechnic University of Catalonia, Barcelona, Spain. Contact him at ovendrel@uci.edu.

RAMIN BOSTANABAD is an associate professor at the University of California, Irvine, Irvine, CA, 92697, USA. His research interests include uncertainty quantification, scientific machine learning, design under uncertainty, and computational mechanics. Bostanabad received his Ph.D. degree in mechanical engineering from Northwestern University. Contact him at Raminb@uci.edu.



Authorized licensed use limited to: Uganda Christian University. Downloaded on October 24,2026 at 14:46:35 UTC from IEEE Xplore. Restrictions apply.